

SQL GROUP BY, LIMIT, and Aggregate Functions

Introduction

SQL provides powerful features to summarize, analyze, and retrieve data efficiently from a database. Three important concepts used in data analysis are:

- **GROUP BY** – Groups rows with similar values.
- **Aggregate Functions** – Perform calculations on groups of data.
- **LIMIT** – Restricts the number of records returned by a query.

These concepts are widely used in reporting, dashboards, analytics, and business intelligence applications.

1. Aggregate Functions

What are Aggregate Functions?

Aggregate functions perform calculations on multiple rows and return a **single value**.

Instead of working on individual rows, they work on an entire column or group of rows.

Common Aggregate Functions

Function	Description
COUNT()	Counts rows
SUM()	Calculates total
AVG()	Calculates average
MAX()	Finds highest value
MIN()	Finds lowest value

Sample Employee Table

Employees

ID	Name	Department	Salary	Age
1	Alice	HR	50000	25
2	Bob	IT	65000	30
3	Charlie	IT	70000	35
4	David	Sales	45000	28
5	Emma	HR	55000	26
6	Frank	Sales	60000	32

COUNT()

Counts the number of rows.

Syntax

```
SELECT COUNT(column_name)
```

```
FROM table_name;
```

Example 1

Count all employees.

```
SELECT COUNT(*) AS TotalEmployees
```

```
FROM Employees;
```

Output

TotalEmployees
6

| 6 |
+-----+

Example 2

Count employees in IT.

```
SELECT COUNT(*)  
FROM Employees  
WHERE Department='IT';
```

Output

2

SUM()

Adds all values of a numeric column.

Syntax

```
SELECT SUM(column_name)  
FROM table_name;
```

Example

```
SELECT SUM(Salary) AS TotalSalary  
FROM Employees;
```

Output

345000

AVG()

Calculates average value.

Syntax

```
SELECT AVG(column_name)  
  
FROM table_name;
```

Example

```
SELECT AVG(Salary) AS AverageSalary  
  
FROM Employees;
```

Output

57500

MAX()

Returns the largest value.

Example

```
SELECT MAX(Salary)  
  
FROM Employees;
```

Output

70000

MIN()

Returns the smallest value.

Example

```
SELECT MIN(Salary)  
  
FROM Employees;
```

Output

45000

Multiple Aggregate Functions Together

SQL allows multiple aggregate functions in one query.

Example

```
SELECT  
COUNT(*) AS Employees,  
SUM(Salary) AS TotalSalary,  
AVG(Salary) AS AverageSalary,  
MAX(Salary) AS HighestSalary,  
MIN(Salary) AS LowestSalary  
FROM Employees;
```

Output

Employees : 6

TotalSalary : 345000

AverageSalary : 57500

HighestSalary : 70000

LowestSalary : 45000

2. GROUP BY Clause

What is GROUP BY?

GROUP BY groups rows having the same values into one summary row.

Instead of calculating on the whole table, aggregate functions calculate for **each group**.

Syntax

```
SELECT column_name,  
aggregate_function(column_name)  
FROM table_name
```

GROUP BY column_name;

Example 1

Find total salary department-wise.

```
SELECT Department,  
SUM(Salary) AS TotalSalary  
FROM Employees  
GROUP BY Department;
```

Output

```
+-----+-----+  
| Department | TotalSalary |  
+-----+-----+  
| HR      | 105000    |  
| IT      | 135000    |  
| Sales   | 105000    |  
+-----+-----+
```

Example 2

Count employees in each department.

```
SELECT Department,  
COUNT(*) AS EmployeeCount  
FROM Employees  
GROUP BY Department;
```

Output

```
+-----+-----+  
| Department | EmployeeCount |
```

HR	2	
IT	2	
Sales	2	

Example 3

Average salary by department.

```
SELECT Department,
AVG(Salary) AS AverageSalary
FROM Employees
GROUP BY Department;
```

Output

Department	AverageSalary	
HR	52500	
IT	67500	
Sales	52500	

Example 4

Highest salary in each department.

```
SELECT Department,
MAX(Salary) AS HighestSalary
FROM Employees
GROUP BY Department;
```

Example 5

Lowest salary in each department.

```
SELECT Department,  
MIN(Salary) AS LowestSalary  
FROM Employees  
GROUP BY Department;
```

GROUP BY with Multiple Columns

You can group by more than one column.

Example Table

Sales

Employee	City	Amount
Amit	Pune	2000
Amit	Mumbai	3000
Rahul	Pune	1500
Rahul	Pune	2500

Query

```
SELECT Employee,  
City,  
SUM(Amount)  
FROM Sales  
GROUP BY Employee, City;
```

Output

Employee	City	SUM
----------	------	-----

Amit	Pune	2000
Amit	Mumbai	3000
Rahul	Pune	4000

GROUP BY with ORDER BY

```
SELECT Department,  
AVG(Salary) AS AverageSalary  
FROM Employees  
GROUP BY Department  
ORDER BY AverageSalary DESC;
```

Output

IT

HR

Sales

GROUP BY with HAVING

The HAVING clause filters grouped records.

Unlike WHERE, HAVING works after grouping.

Syntax

```
SELECT column_name,  
aggregate_function(column_name)  
FROM table_name  
GROUP BY column_name  
HAVING condition;
```

Example

Departments having average salary greater than 55,000.

```
SELECT Department,  
AVG(Salary) AS AverageSalary  
FROM Employees  
GROUP BY Department  
HAVING AVG(Salary) > 55000;
```

Output

IT

WHERE vs HAVING

WHERE	HAVING
Filters rows before grouping	Filters groups after grouping
Cannot use aggregate functions	Can use aggregate functions
Executed first	Executed after GROUP BY

Example

```
SELECT Department,  
AVG(Salary)  
FROM Employees  
WHERE Salary > 50000  
GROUP BY Department;
```

Example

```
SELECT Department,  
AVG(Salary)  
FROM Employees
```

GROUP BY Department
HAVING AVG(Salary)>55000;

3. LIMIT Clause

What is LIMIT?

LIMIT restricts the number of rows returned.

Useful for:

- Top records
 - Pagination
 - Previewing data
 - Performance optimization
-

Syntax

SELECT columns
FROM table_name
LIMIT number;

Example 1

Display first three employees.

```
SELECT *  
FROM Employees  
LIMIT 3;
```

Output

Alice

Bob

Charlie

Example 2

Display first two departments.

```
SELECT Department  
FROM Employees  
LIMIT 2;
```

LIMIT with ORDER BY

Usually used together.

Example

Highest-paid employee.

```
SELECT *  
FROM Employees  
ORDER BY Salary DESC  
LIMIT 1;
```

Output

Charlie
70000

Example

Top three highest salaries.

```
SELECT Name,  
Salary  
FROM Employees  
ORDER BY Salary DESC  
LIMIT 3;
```

Output

Charlie

Bob

Frank

LIMIT with OFFSET

OFFSET skips rows.

Syntax

```
SELECT *
```

```
FROM Employees
```

```
LIMIT 2 OFFSET 3;
```

Output

Fourth and Fifth Records

Pagination Example

Page size = 5

First page

```
SELECT *
```

```
FROM Employees
```

```
LIMIT 5 OFFSET 0;
```

Second page

```
SELECT *
```

```
FROM Employees
```

```
LIMIT 5 OFFSET 5;
```

Third page

```
SELECT *
```

```
FROM Employees  
LIMIT 5 OFFSET 10;
```

Complete Example

```
SELECT Department,  
COUNT(*) AS Employees,  
AVG(Salary) AS AverageSalary,  
MAX(Salary) AS HighestSalary,  
MIN(Salary) AS LowestSalary,  
SUM(Salary) AS TotalSalary  
FROM Employees  
GROUP BY Department  
HAVING AVG(Salary) > 50000  
ORDER BY TotalSalary DESC  
LIMIT 3;
```

This query:

- Groups employees by department.
 - Counts employees.
 - Calculates average salary.
 - Finds highest salary.
 - Finds lowest salary.
 - Calculates total salary.
 - Filters departments with average salary greater than 50,000.
 - Sorts by total salary in descending order.
 - Displays only the top 3 departments.
-

Practice Programs

Program 1: Count Students in Each Course

```
SELECT Course,  
COUNT(*) AS TotalStudents  
FROM Students  
GROUP BY Course;
```

Program 2: Find Maximum Salary

```
SELECT MAX(Salary)  
FROM Employees;
```

Program 3: Average Marks Subject-wise

```
SELECT Subject,  
AVG(Marks)  
FROM StudentMarks  
GROUP BY Subject;
```

Program 4: Top Five Expensive Products

```
SELECT ProductName,  
Price  
FROM Products  
ORDER BY Price DESC  
LIMIT 5;
```

Program 5: Departments Having More Than Two Employees

```
SELECT Department,  
COUNT(*) AS EmployeeCount  
FROM Employees  
GROUP BY Department
```

HAVING COUNT(*) > 2;

Program 6: Total Sales by City

```
SELECT City,  
SUM(Amount) AS TotalSales  
FROM Sales  
GROUP BY City;
```

Program 7: Top Three Students by Marks

```
SELECT Name,  
Marks  
FROM Students  
ORDER BY Marks DESC  
LIMIT 3;
```

Summary

Concept	Purpose
---------	---------

COUNT ()	Counts rows
-----------	-------------

SUM ()	Calculates total values
---------	-------------------------

AVG ()	Calculates average
---------	--------------------

MAX ()	Finds highest value
---------	---------------------

MIN ()	Finds lowest value
---------	--------------------

GROUP BY	Groups rows with the same values
HAVING	Filters grouped data
LIMIT	Restricts the number of rows returned
OFFSET	Skips a specified number of rows
ORDER BY	Sorts the results before applying LIMIT

Key Points to Remember

- Aggregate functions return a single summarized value.
- **GROUP BY** is used with aggregate functions to summarize data for each group.
- **WHERE** filters individual rows before grouping, while **HAVING** filters groups after aggregation.
- **ORDER BY** is commonly combined with **LIMIT** to retrieve top or bottom records.
- **LIMIT** with **OFFSET** is useful for implementing pagination in applications.
- Aggregate functions ignore **NULL** values (except **COUNT (*)**, which counts all rows).